

Session 6 — Smart Contract Programming I

Buổi 6 — Lập trình hợp đồng thông minh I

Solidity & Hardhat

Blockchain Technology and Cryptocurrency

TS. Nguyễn Trung Thành

Contract anatomy • types • events & errors • Hardhat • deploy to TrustKeys

Objectives / Mục tiêu

1. Read & write a Solidity contract: state, visibility, `view/pure/payable`, events, custom errors, modifiers
Đọc & viết hợp đồng Solidity: trạng thái, phạm vi, sự kiện, lỗi tùy biến, modifier
2. Pick the right type & data location (`storage/memory/calldata`) and reason about **gas**
*Chọn đúng kiểu & vị trí dữ liệu và suy luận về **gas***
3. Run the **Hardhat** loop: compile → test (Chai + `loadFixture`) → deploy → interact
Chạy vòng lặp Hardhat: biên dịch → test → deploy → tương tác
4. Deploy to the **TrustKeys L1 testnet** and read state back with ethers (no explorer)
Triển khai lên testnet TrustKeys và đọc trạng thái bằng ethers (không có explorer)
5. **Register your final-project team & topic** (đăng ký nhóm & đề tài đồ án cuối kỳ)

Agenda — 3 hours / 3 giờ

Time	Content / Nội dung
0:00–0:55	Solidity language — ngôn ngữ Solidity
0:55–1:05	Break / <i>giải lao</i>
1:05–2:00	Hardhat + Remix: test, deploy, interact
2:00–2:10	Break / <i>giải lao</i>
2:10–3:05	Lab <code>ClassRegistry</code> on TrustKeys + project registration

Recap / Ôn lại (Session 5)

- The **EVM** is a world computer; accounts = **EOA** vs **contract**
EVM là máy tính toàn cầu; tài khoản = EOA vs hợp đồng
- Every state change costs **gas**; fees follow **EIP-1559** (base + priority)
Mọi thay đổi trạng thái tốn gas; phí theo EIP-1559
- A contract = **code + storage** living at an address
Hợp đồng = mã + bộ nhớ trạng thái sống tại một địa chỉ

Today we write that code. *Hôm nay ta viết đoạn mã đó.*

Hour 1 / Giờ 1

The Solidity language / Ngôn ngữ Solidity

From "hello contract" to events, errors & gas

What is a smart contract? / Hợp đồng thông minh là gì?

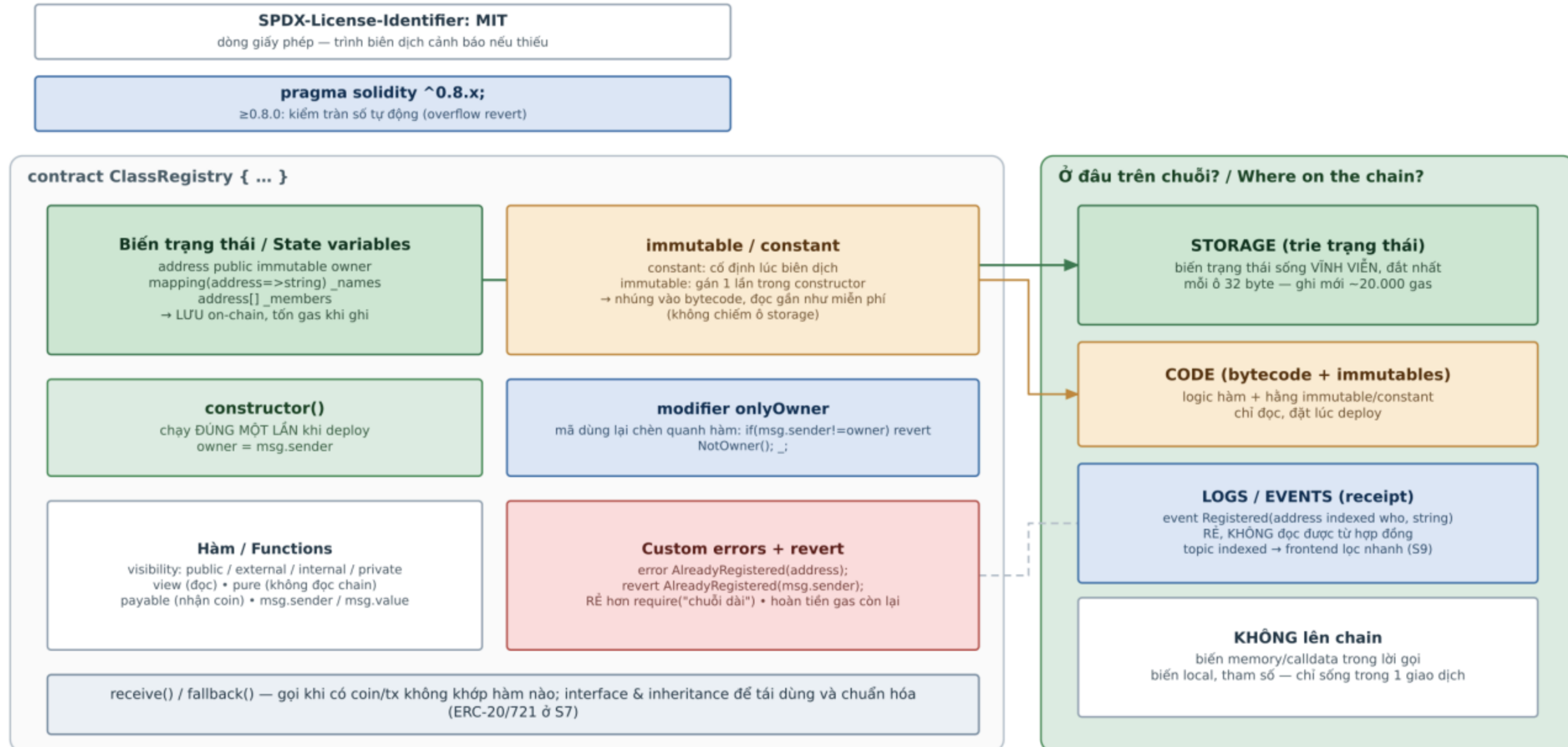
- A program deployed at an address; its **public functions are its API**
Chương trình triển khai tại một địa chỉ; hàm public là API của nó
- **Deterministic**: every node re-executes it and must agree
Tất định: mọi node thực thi lại và phải cùng kết quả
- **Immutable by default**: code you ship is (mostly) the code forever — bugs are permanent
Bất biến mặc định: mã đã phát hành gần như là mãi mãi — lỗi tồn tại vĩnh viễn
- Language: **Solidity** (statically typed, compiles to EVM bytecode)
Ngôn ngữ: Solidity (kiểu tĩnh, biên dịch ra bytecode EVM)

Rule / Nguyên tắc: test *before* you deploy — you cannot patch in place.
Kiểm thử TRƯỚC khi deploy — không sửa tại chỗ được.

Anatomy of a contract / Giải phẫu hợp đồng

Giải phẫu một hợp đồng Solidity — Anatomy of a Solidity contract

Cái gì nằm on-chain vĩnh viễn (storage) vs cái gì chỉ là mã/nhật ký • What lives on-chain forever (storage) vs code and logs



The three header lines / Ba dòng đầu

```
// SPDX-License-Identifier: MIT           // license tag / thẻ giấy phép
pragma solidity ^0.8.24;                   // compiler range / dải trình biên dịch

contract ClassRegistry { /* ... */ }
```

- **SPDX**: machine-readable license; compiler warns if missing
Giấy phép máy đọc được; thiếu sẽ bị cảnh báo
- **^0.8.24** : any **0.8.x** $\geq 0.8.24$, **< 0.9.0**
^0.8.24 : mọi 0.8.x $\geq 0.8.24$ và $< 0.9.0$
- **Since 0.8.0**: arithmetic **overflow/underflow reverts automatically** — no more SafeMath
*Từ 0.8.0: tràn số **tự động revert** — không cần SafeMath nữa*

State variables / Biến trạng thái

```
address public immutable owner;           // in bytecode, not a storage slot
mapping(address => string) private _names; // lives in storage forever
address[] private _members;
```

- Declared at contract level → **persist in on-chain storage** between txs
Khai báo ở cấp hợp đồng → tồn tại trong storage giữa các giao dịch
- `public` auto-creates a **getter** function (*tự sinh hàm getter*)
- **Writing storage is the most expensive thing you do** (*ghi storage là thao tác đắt nhất*)

Constructor & message globals / Constructor & biến toàn cục

```
constructor() {  
    owner = msg.sender;    // whoever deploys becomes owner  
}
```

Global	Meaning / Ý nghĩa
<code>msg.sender</code>	address that called this function / <i>người gọi hàm</i>
<code>msg.value</code>	wei sent with the call / <i>số wei gửi kèm</i>
<code>block.timestamp</code>	block time (seconds) / <i>thời gian block (giây)</i>
<code>block.number</code>	current block height / <i>chiều cao block</i>

constructor runs exactly once, at deploy. (*chạy đúng một lần khi deploy*)

⚠ **block.timestamp caveats / Cảnh báo**

- Set by the **block proposer**, not a trusted clock — can be nudged a few seconds
Do người đề xuất block đặt, không phải đồng hồ tin cậy — lệch được vài giây
- **Never** use it as a randomness source (miners/proposers can bias it)
Tuyệt đối không dùng làm nguồn ngẫu nhiên
- Fine for **coarse** deadlines (days), dangerous for **fine** timing or lotteries
Ổn cho hạn "thô" (ngày), nguy hiểm cho hạn giờ "mịn" hay xổ số

Story / Câu chuyện: early gambling dApps drained by timestamp/ `blockhash` "randomness".

Nhiều dApp cờ bạc đời đầu bị rút sạch vì "ngẫu nhiên" từ timestamp/blockhash.

Visibility / Phạm vi truy cập

Keyword	Callable from / Gọi được từ
public	anywhere (external + internal) / <i>mọi nơi</i>
external	only outside calls (cheaper for big args) / <i>chỉ từ ngoài</i>
internal	this contract + children / <i>hợp đồng này + kế thừa</i>
private	this contract only / <i>chỉ hợp đồng này</i>

⚠ `private` $\neq$ secret: **all state is readable off-chain** (*mọi trạng thái đều đọc được ngoài chuỗi* — *private chỉ giới hạn ở tầng ngôn ngữ*)

view, pure, payable / Ba từ khóa hàm

```
function isRegistered(address who) public view returns (bool) { ... } // reads state
function add(uint a, uint b) public pure returns (uint) { return a + b; } // no state
function deposit() public payable { ... } // may receive coin
```

- **view** — reads but never writes state; **free** when called off-chain (`eth_call`)
đọc, không ghi; miễn phí khi gọi ngoài chuỗi
- **pure** — touches no state at all (*không đụng trạng thái*)
- **payable** — the function may receive `msg.value` (*có thể nhận coin*)

Value vs reference types / Kiểu giá trị vs tham chiếu

Value / Giá trị (copied on assignment): `uint256`, `int`, `bool`, `address`, `bytes32`, `enum`
sao chép khi gán

Reference / Tham chiếu (need a data location): `string`, `bytes`, `arrays`, `struct`,
`mapping`
cần khai báo vị trí dữ liệu

```
uint256 a = 5; uint256 b = a;    // b is an independent copy
```

- `address` VS `address payable` — only the latter can `.transfer/.send`
chỉ `address payable` mới `.transfer/.send` được
- **Units / Đơn vị:** `1 gwei` = 10^9 wei (gas), `1 ether` = 10^{18} wei → `msg.value == 0.01 ether`
`1 gwei` = 10^9 wei, `1 ether` = 10^{18} wei

Mappings / Ánh xạ

```
mapping(address => string) private _names;    // key → value
_names[msg.sender] = name;                    // 0(1) write
```

- Every possible key exists, mapped to the **zero value** by default
Mọi khóa đều "tồn tại", giá trị mặc định = giá trị 0
- **You cannot iterate a mapping** — no length, no keys list
Không duyệt được mapping — không có độ dài, không liệt kê khóa
- Pattern: keep a parallel `address[]` **array** to enumerate
Mẫu: giữ thêm mảng `address[]` để duyệt → the `ClassRegistry` `_members` list

Arrays, structs, enums / Mảng, struct, enum

```
struct Member { string name; uint256 joinedAt; } // custom record
enum Status { Pending, Active, Removed } // named constants 0,1,2

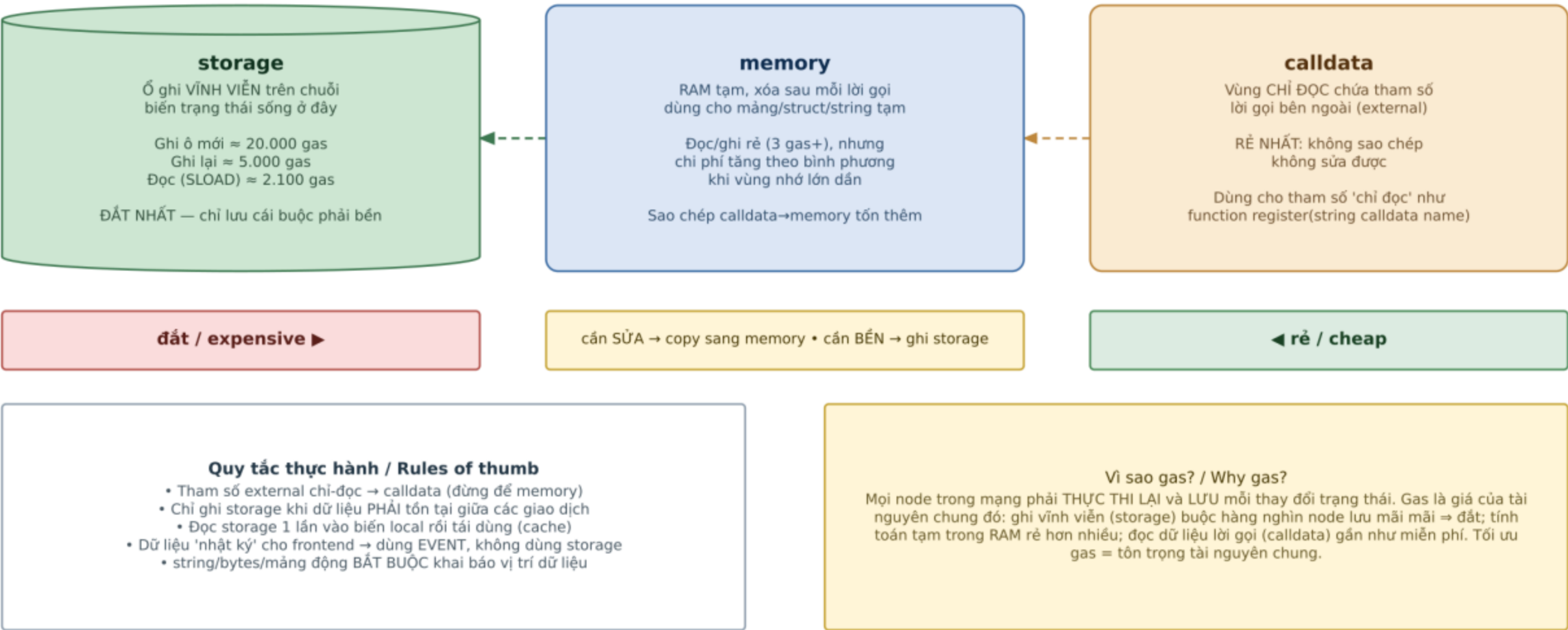
Member[] public members; // dynamic array
mapping(address => Status) public status;
```

- Dynamic arrays: `.push()`, `.pop()`, `.length` (*mảng động*)
- `enum` = readable small integers (*số nguyên nhỏ có tên*)
- **swap-and-pop** to delete from an array cheaply (*xóa phần tử mảng rẻ*) — used in the lab

Data location & gas / Vị trí dữ liệu & gas

Vị trí dữ liệu & trực giác về gas — Data location & gas intuition

storage = đĩa cứng vĩnh viễn (đắt) • memory = RAM tạm • calldata = chỉ đọc, rẻ nhất • storage = permanent disk, memory = scratch RAM, calldata = read-only cheapest



calldata vs memory in practice / Trong thực tế

```
function register(string calldata name) external {    // read-only arg → calldata
    _names[msg.sender] = name;                        // copy into storage
}
```

- **calldata** for external read-only params → **cheapest**, no copy
tham số external chỉ đọc → rẻ nhất
- **memory** when you must modify a temporary copy (*khi cần sửa bản tạm*)
- **storage** references point at persistent state (*tham chiếu tới trạng thái bền*)

Rule / Quy tắc: don't write to **storage** unless the data must outlive the tx.
Đừng ghi storage trừ khi dữ liệu phải sống qua giao dịch.

Events / Sự kiện

```
event Registered(address indexed who, string name);  
emit Registered(msg.sender, name);
```

- Written to the tx **log** (receipt), **not** contract storage → **cheap**
Ghi vào log của giao dịch, không phải storage → rẻ
- **indexed** fields become searchable **topics** (up to 3) — frontends filter fast
*trường **indexed** thành "topic" tìm kiếm được — frontend lọc nhanh*
- Contracts **cannot read** their own events; off-chain code can (preview **S9**)
Hợp đồng không đọc lại event của mình; mã ngoài chuỗi đọc được (xem trước S9)

Custom errors vs require / Lỗi tùy biến

```
error AlreadyRegistered(address who);    // declare once

if (_indexPlusOne[msg.sender] != 0)
    revert AlreadyRegistered(msg.sender); // cheaper than a string
```

Style	Cost / Chi phí
<code>require(cond, "long reason string")</code>	stores the string → more gas / <i>tốn hơn</i>
<code>revert CustomError(args)</code>	4-byte selector + args → cheaper / <i>rẻ hơn</i>

- `revert` **undoes all state changes** in the tx and refunds remaining gas
revert hoàn tác mọi thay đổi trong giao dịch và hoàn gas còn lại

Modifiers / Modifier

```
modifier onlyOwner() {  
    if (msg.sender != owner) revert NotOwner();  
    _; // <- the function body runs here  
}  
  
function deregister(address who) external onlyOwner { ... }
```

- Reusable **guard code** wrapped around functions (*mã bảo vệ tái dùng quanh hàm*)
- Classic uses: access control, pause switch, reentrancy guard (S8)
Dùng cho: kiểm soát truy cập, khóa tạm, chống reentrancy (S8)
- The `_;` marks where the wrapped body executes (`_;` là nơi thân hàm chạy)

receive & fallback / receive & fallback

```
receive() external payable { }           // plain coin transfer, empty data
fallback() external payable { }          // unknown selector or leftover cases
```

- **receive** runs on a bare coin send (no calldata)
chạy khi nhận coin trơn (không calldata)
- **fallback** runs when no function matches — or as a proxy hook
chạy khi không hàm nào khớp — hoặc làm móc proxy
- No **payable** receive/fallback → contract **rejects** plain transfers
không có receive/fallback payable → hợp đồng từ chối coin trơn

Inheritance, interfaces, constants / Kế thừa, interface, hằng

```
interface IERC20 { function transfer(address to, uint256 v) external returns (bool); }
contract Token is Ownable { ... }           // `is` = inherit

uint256 constant MAX = 100;                 // fixed at compile time
address immutable OWNER;                    // set once in constructor
```

- **Interfaces** = shared shape → how contracts talk (ERC-20/721 in **S7**)
Interface = "khuôn" chung → cách các hợp đồng nói chuyện
- **constant** / **immutable** live in **bytecode**, not storage → near-free reads
sống trong bytecode, đọc gần như miễn phí
- Reuse audited code: **OpenZeppelin** (*tái dùng mã đã kiểm toán*)

Break 10' / Giải lao

Next / *tiếp theo*: from source file to a live contract on TrustKeys

Từ tệp mã tới hợp đồng chạy thật trên TrustKeys

Hour 2 / Giờ 2

Hardhat + Remix: test, deploy, interact

Kiểm thử, triển khai, tương tác

Two dev environments / Hai môi trường

	Remix	Hardhat
Install / Cài	none — browser / <i>không cần</i>	Node ≥ 18 + npm
Best for / Hợp cho	first contact, quick deploy	real projects, CI, testing
Tests / Kiểm thử	manual	automated JS/Chai
URL	<code>remix.trustkeys.com</code>	local CLI

Plan / Kế hoạch: meet Solidity in **Remix**, then adopt **Hardhat** as the project toolchain.
Làm quen trong Remix, rồi dùng Hardhat làm bộ công cụ dự án.

Remix — first contact / Làm quen

Hosted, pre-wired for TrustKeys: `https://remix.trustkeys.com`

1. New file `ClassRegistry.sol` → paste the contract (*dán hợp đồng*)
2. **Solidity Compiler** tab → compile (pick `0.8.24+`) (*biên dịch*)
3. **Deploy & Run** tab → Environment = **Injected Provider - MetaMask**
4. MetaMask on **TrustKeys** (RPC `l1testnet.trustkeys.network`, chainId **11968**)
5. **Deploy** → confirm in MetaMask → call functions from the UI
Deploy → xác nhận trong MetaMask → gọi hàm từ giao diện

Great for beginners — see a contract live in 5 minutes. (*thấy hợp đồng chạy trong 5 phút*)

Hardhat — install & init / Cài & khởi tạo

```
mkdir classregistry && cd classregistry
npm init -y
npm i --save-dev hardhat @nomicfoundation/hardhat-toolbox dotenv
npx hardhat init          # choose: JavaScript project
```

- **hardhat** — the task runner & local EVM (*bộ chạy tác vụ + EVM cục bộ*)
- **hardhat-toolbox** — ethers, Chai matchers, network-helpers, gas reporter...
- **dotenv** — load secrets from **.env** (*nạp bí mật từ .env*)
- Requires **Node ≥ 18** (*yêu cầu Node ≥ 18*)

Project layout / Bố cục dự án

```
classregistry/  
├── contracts/      ClassRegistry.sol      # your Solidity  
├── test/           ClassRegistry.test.js   # Chai tests  
├── scripts/        deploy.js              # deploy & interact  
├── hardhat.config.js # networks, compiler  
└── .env            PRIVATE_KEY=...        # git-ignored!
```

- `npx hardhat compile` → artifacts (ABI + bytecode) in `artifacts/`
sinh ABI + bytecode
- Ignition (`ignition/`) is an alternative to scripts for declarative deploys
Ignition là lựa chọn thay cho scripts để deploy khai báo

Testing culture from day one / Văn hóa kiểm thử

```
const { expect } = require("chai");
const { loadFixture } = require("@nomicfoundation/hardhat-network-helpers");

it("stores the name", async function () {
  const { registry, alice } = await loadFixture(deployRegistryFixture);
  await registry.connect(alice).register("Alice");
  expect(await registry.nameOf(alice.address)).to.equal("Alice");
});
```

- Tests run on an **in-memory chain**: no gas, no waiting, resets each run
Chạy trên chain trong bộ nhớ: không gas, không chờ, reset mỗi lần
- **Write the test with the contract**, not after (*viết test cùng lúc với hợp đồng*)

loadFixture & signers / loadFixture & tài khoản

```
async function deployRegistryFixture() {  
  const [owner, alice, bob] = await ethers.getSigners(); // 20 funded test accounts  
  const Registry = await ethers.getContractFactory("ClassRegistry");  
  const registry = await Registry.deploy(); // owner = signer[0]  
  return { registry, owner, alice, bob };  
}
```

- `getSigners()` → 20 pre-funded local accounts (*20 tài khoản có sẵn tiền*)
- `loadFixture` deploys **once**, then snapshot-reverts → fast & independent tests
deploy một lần rồi quay lui ảnh chụp → test nhanh & độc lập
- `registry.connect(alice)` sends the next call **as** alice (*gọi với tư cách alice*)

Asserting events, errors, time / Kiểm event, lỗi, thời gian

```
await expect(registry.connect(alice).register("Alice"))
  .to.emit(registry, "Registered").withArgs(alice.address, "Alice");

await expect(registry.connect(alice).register("again"))
  .to.be.revertedWithCustomError(registry, "AlreadyRegistered")
  .withArgs(alice.address);

await time.increase(3600); // fast-forward 1 hour on the local chain
```

- `.to.emit(...).withArgs(...)` — assert a log fired (*khẳng định event bắn ra*)
- `.to.be.revertedWithCustomError(...)` — assert the right revert (*đúng lỗi*)
- `hardhat-gas-reporter` prints gas per function (*in gas mỗi hàm*)

Local chain & console / Chain cục bộ & console

```
npx hardhat node          # a JSON-RPC chain on localhost:8545 + 20 accounts  
npx hardhat console --network localhost # interactive REPL
```

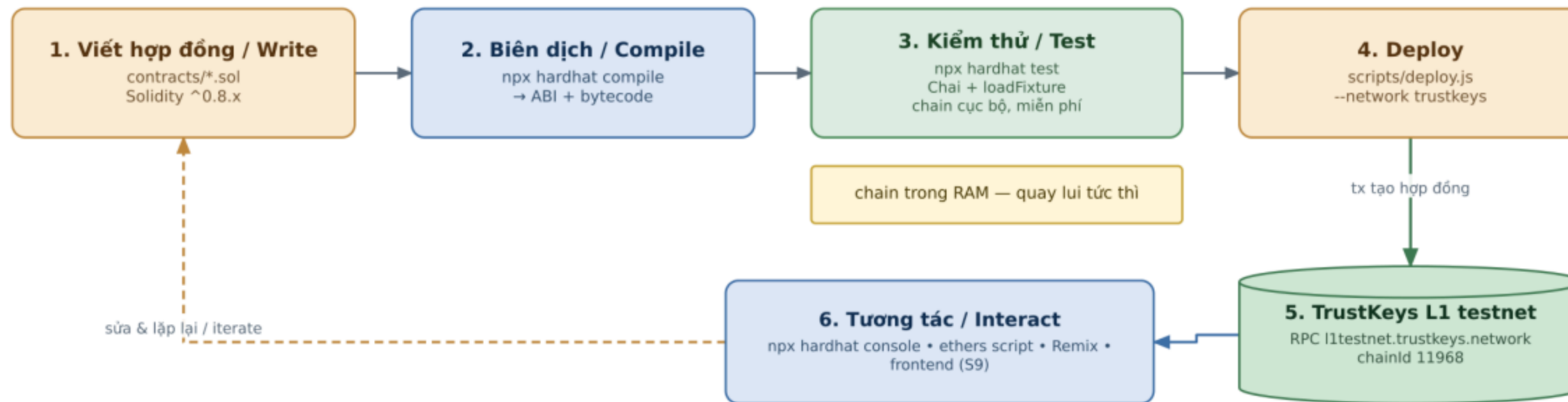
```
> const r = await ethers.getContractAt("ClassRegistry", addr);  
> await r.memberCount();
```

- `npx hardhat node` = full local blockchain for manual poking / a frontend
blockchain cục bộ đầy đủ để thử tay / cắm frontend
- `console` = live ethers session against any network (*phiên ethers trực tiếp*)

The Hardhat loop / Vòng lặp Hardhat

Vòng lặp phát triển với Hardhat — The Hardhat development loop

Viết → test cục bộ (miễn phí, tức thì) → deploy → TrustKeys testnet → tương tác • Write → test locally (free, instant) → deploy → TrustKeys testnet → interact



Test chạy trên chain Hardhat trong bộ nhớ: KHÔNG tốn phí, KHÔNG chờ block, reset mỗi lần — nên chạy hàng trăm test/giây. Chỉ deploy lên testnet khi test đã xanh. Tests run on an in-memory Hardhat chain: no gas, no block wait, resets each run — run hundreds of tests/second. Deploy to the testnet only after tests are green. TrustKeys KHÔNG có explorer công khai → đọc trạng thái bằng lời gọi ethers (view + queryFilter), không phải web explorer.

Config & key safety / Cấu hình & an toàn khóa

```
// hardhat.config.js
networks: {
  trustkeys: {
    url: "https://l1testnet.trustkeys.network",
    chainId: 11968,
    accounts: [process.env.PRIVATE_KEY], // from .env
  },
},
```

-  `PRIVATE_KEY` = a dedicated MetaMask TEST account — NEVER a real key
khóa của một tài khoản MetaMask THỬ riêng — TUYỆT ĐỐI không phải khóa thật
- `.env` is **git-ignored**; commit `.env.example` only
`.env` bị git bỏ qua; chỉ commit `.env.example`

Deploy to TrustKeys / Triển khai lên TrustKeys

```
// scripts/deploy.js
const Registry = await ethers.getContractFactory("ClassRegistry");
const registry = await Registry.deploy();
await registry.waitForDeployment();
console.log("deployed at:", await registry.getAddress());
```

```
npx hardhat run scripts/deploy.js --network trustkeys
```

- Uses your funded test account; the deploy is a real transaction on the testnet
Dùng tài khoản thử đã được nạp tiền; deploy là một giao dịch thật trên testnet
- **Save the printed address** *(lưu lại địa chỉ in ra)*

Interact — no explorer / Tương tác — không explorer

TrustKeys has **no public block explorer** → read state via **ethers**.

TrustKeys không có explorer công khai → đọc trạng thái bằng ethers.

```
const r = await ethers.getContractAt("ClassRegistry", addr);
await r.memberCount(); // view call
const evs = await r.queryFilter(r.filters.Registered()); // event log
evs.forEach(e => console.log(e.args.who, e.args.name));
```

- `queryFilter` replays the **event log** — exactly how a frontend indexes data
`queryFilter` phát lại log sự kiện — đúng cách frontend đánh chỉ mục
- Options: Hardhat **console**, a read **script**, or the **Remix** UI (*console, script, hoặc Remix*)

Break 10' → Lab & Project

Have ready / *chuẩn bị*: MetaMask on TrustKeys, `npm i` done, funded test address
MetaMask trên TrustKeys, đã `npm i`, địa chỉ thử đã có tiền

Lab 6 — ClassRegistry (50') / Thực hành

Build, test, deploy a real contract to TrustKeys.

Xây, kiểm thử, triển khai một hợp đồng thật lên TrustKeys.

	Step / Bước	Deliverable / Sản phẩm
10	<code>npx hardhat init</code> (JS) + run sample test	sample test passes / <i>test mẫu qua</i>
20	write <code>ClassRegistry.sol</code> + Chai tests	all tests green / <i>test xanh</i>
15	deploy <code>--network trustkeys</code> + register	on-chain address / <i>địa chỉ on-chain</i>
5	read classmates via <code>queryFilter</code>	list of names / <i>danh sách tên</i>

Files / *tệp*: `lab/lab06_worksheet.md`, `lab/solutions/`. The `ClassRegistry` exercises Hour 1: `mapping` + `address[]` + swap-and-pop, `immutable owner`, `onlyOwner`, custom error, indexed event.

🎓 Final Project — registration opens today / Đồ án cuối kỳ

Teams of 2-3 • topic + team due end of week. (*Nhóm 2-3 • đề tài + nhóm hạn cuối tuần.*)

- **Weight / Trọng số:** 60% of the course = $0.6 \times \text{Project (group)} + 0.4 \times \text{Oral defense}$
60% học phần = $0.6 \times \text{Đồ án nhóm} + 0.4 \times \text{Vấn đáp cá nhân}$
- **Deliver / Nộp:** Git repo (contracts + tests + frontend) • deployed & verified DApp • 10-15p report • ≤ 5 -min demo • contribution log
Repo Git, DApp đã deploy & verify, báo cáo 10-15 trang, video demo, nhật ký đóng góp
- **Constraints / Ràng buộc:** testnet only; you must explain **every line** at the oral defense (AI-assisted code allowed but **declared**)
Chỉ testnet; phải giải thích MỌI DÒNG khi vấn đáp (được dùng AI nhưng phải khai báo)

Topics 1-5 / Đề tài 1-5

#	Topic	Minimum / Tối thiểu	★ Extension
1	Token + Staking	ERC-20 capped + time-reward staking + UI	permit (EIP-2612); invariant tests
2	NFT Marketplace	ERC-721 + IPFS; list/buy/cancel escrow; royalty	auctions; lazy mint; The Graph
3	DAO Governance	gov token + Governor; propose/vote/execute	timelock; quadratic/delegated vote
4	Mini-AMM DEX	constant-product pool; add/remove/swap+fee	router; TWAP oracle; LP NFT
5	Crowdfunding	goal/deadline/refund-if-fail; ETH or ERC-20	milestone vesting; KYC allowlist

Topics 6-10 / Đề tài 6-10

#	Topic	Minimum / Tối thiểu	★ Extension
6	On-chain Certificates	issuer registry; soulbound NFT; verify page	ZK proof-of-degree; Merkle batch
7	Supply-chain Trace	lifecycle events (ERC-1155); QR verify	role-based flow; L2 + cost analysis
8	Stablecoin Gateway	merchant invoices; pay; status indexer	AA checkout; split payments
9	Prediction Market	binary create/bet/resolve/claim; oracle	LMSR/CPMM pricing; dispute window
10	Token-gated Content	NFT membership; SIWE; gated server check	expiring subs; cross-chain check

Topics 11-14 / Đề tài 11-14

#	Topic	Minimum / Tối thiểu	★ Extension
11	ZK Application	Circom/Noir circuit; on-chain verifier; UI	nullifiers; anon voting/airdrop
12	AI Agent with Wallet	LLM agent + testnet wallet + spend-limit contract	session keys (AA); m2m payment
13	Lending Micro-protocol	deposit/borrow; health factor; liquidation	rate model; oracle + fork tests
14	On-chain Game	commit-reveal RPS with wagers	VRF randomness; leaderboard NFT

Own topic? Allowed with instructor approval. *(Tự đề xuất? Được, cần GV duyệt.)*

Timeline & grading / Mốc thời gian & chấm điểm

Milestone / Mốc	When / Khi nào
Topic + team registration / Đăng ký đề tài + nhóm	S6 → end of week
2-page design (pass/fail gate) / Thiết kế 2 trang	S10
Workshop + peer review / Xưởng + đánh giá chéo	S14
Live demo / Demo trực tiếp	S15
Final package + oral defense / Nộp + vấn đáp	Exam week

Rubric / *Rubric*: Functionality 2.5 • Contracts & testing 2.0 • Security 1.5 • Frontend 1.5 • Modern-stack 1.0 • Report 1.0 • Demo 0.5

Register your team / Đăng ký nhóm

Form fields / Trường biểu mẫu: team name • 2-3 members (name + ID) • topic # (1-14 or proposal) • one-line idea • repo URL (later)

Tên nhóm • 2-3 thành viên • số đề tài • ý tưởng một dòng • URL repo (sau)

- Balance teams: pair a stronger coder with a weaker one (*cân bằng nhóm*)
- **Weaker teams:** topics 1, 5, 6, 14 (*nền tảng vững, ít phụ thuộc*)
- **Stronger teams:** topics 3, 4, 9, 11, 12, 13 (*oracle/ZK/AA/AMM khó hơn*)
- Link in the class channel — **due end of week** (*link trong kênh lớp — hạn cuối tuần*)

Homework / Bài về nhà (trước Buổi 7)

1. Finish `ClassRegistry`: $\geq$ **6 passing tests**, deploy to TrustKeys, paste your address + tx hash
Hoàn thiện: $\geq$ 6 test qua, deploy, dán địa chỉ + tx hash
2. Register your **team & topic** in the form (*đăng ký nhóm & đề tài*)
3. Add one extra `view` function + a test for it (*thêm 1 hàm view + test*)
4. Read / *Đọc*: docs.soliditylang.org (types, units) • hardhat.org tutorial

Next / Buổi sau

Session 7 — Token Standards

ERC-20 • ERC-721 • ERC-1155 • on-chain metadata

You built a registry; next you build **money and NFTs** with OpenZeppelin.

*Bạn vừa xây registry; tiếp theo xây **token và NFT** với OpenZeppelin.*

References / Tài liệu tham khảo

- **Solidity docs** — docs.soliditylang.org (language, types, units, error handling)
- **Hardhat docs** — hardhat.org/docs (getting started, testing, network config)
- **Ethers v6** — docs.ethers.org (contracts, `queryFilter`, providers)
- **Chai matchers / network-helpers** — hardhat.org/hardhat-chai-matchers, [/hardhat-network-helpers](https://hardhat.org/hardhat-network-helpers)
- **OpenZeppelin Contracts** — docs.openzeppelin.com/contracts
- **Ethereum dev portal** — ethereum.org/developers (smart-contract anatomy, gas)
- **EIP-1559** (fees), **EIP-2612** (permit) — eips.ethereum.org
- **Remix (TrustKeys)** — remix.trustkeys.com • **TrustKeys L1**: RPC `l1testnet.trustkeys.network`, chainId 11968